

ADVANCED SQL

ZADATAK 1

Ispisati godinu (vremenskog trenutka *trackView.viewStartDateTime*), ime i prezime vlasnika korisničkog računa i broj **filmova** pokrenutih od strane svih profila korisničkog računa u toj godini.

Dodatno ispisati kumulativni broj pokretanja filmova (*cntThisAndPrevYear*), koji obuhvaća i pokretanja iz svih prethodnih godina.

U obzir uzeti samo filmove koje je režirao 'Jan Suter' ili 'Jay Chapman'.

Donja tablica ilustrira izgled rezultata.

```
select extract (year from trackview.viewStartDateTime) as viewyear, firstname, lastname,
count(trackview.ownerid) as cntinyear,
sum(count(trackview.ownerid)) over (partition by trackview.ownerid order by extract (year from trackview.viewStartDateTime)
rows between unbounded preceding and current row) as thisandprevyears
from trackview left join track on trackview.trackid = track.trackid
left join owner on trackview.ownerid = owner.ownerid
where director @> array['Jan Suter']::varchar[] or director @> array['Jay Chapman']::varchar[]
group by viewyear, firstname, lastname, trackview.ownerid
order by firstname, lastname, viewyear
```

ZADATAK 2

Za filmove koji u **nazivu** sadrže riječ nalik riječi '**magia**' ispisati naziv filma te ime i prezime osobe koja je navedena kao **prva** od svih koje su sudjelovale u stvaranju filma (*castAndCrew*).

Nije potrebno ispisati podatke za filmove za koje nije navedena nijedna (pa ni prva) osoba u stvaranju filma.

Nalikuje li riječ iz naslova filma riječi '**magia**' ispitajte pomoću implementacije Q-gram algoritma u PostgreSQL-u.

```
SELECT tracktitle, castandcrew[1] AS firstcastcrew
FROM track,
UNNEST(string_to_array(tracktitle, ' ')) AS words(w)
WHERE similarity(w, 'magia') > 0.3
AND castandcrew[1] IS NOT NULL
```

ZADATAK 3

Za filmove koji u **nazivu** sadrže riječ nalik riječi '**rank**' ispisati naziv filma te naziv prve države od svih država koje su financirale film (*countryOfOrigin*).

Ne treba ispisati naziv filma ako nije navedena nijedna država koja je financirala film.

Nalikuje li riječ iz naslova filma riječi '**rank**' ispitajte pomoću implementacije Q-gram algoritma u PostgreSQL-u.

```
SELECT DISTINCT tracktitle, countryoforigin[1]
FROM track,
UNNEST(string_to_array(tracktitle, ' ')) AS words(w)
WHERE countryoforigin[1] IS NOT NULL
AND similarity(w, 'rank') > 0.3
```

ZADATAK 4

Ispisati različita trajanja **filmova** i ukupan broj profila koji su gledali/pokretali filmove. U obzir uzeti samo filmove koji:

- **bilo u nazivu bilo u opisu** sadrže riječ čiji je normalizirani oblik jednak **'activ'** ili **'worthi'** ili **'ride'**
- pripadaju nekom od žanrova: 'Dramas', 'Comedies', 'Documentaries' i 'Action & Adventure'.

Donja tablica ilustrira izgled rezultata.

duration	action_adventure	comedies	documentaries	dramas
...	...	...	...	...
00:28:00	null	null	2	null
00:40:00	5	null	5	null

```
CREATE TEMP TABLE zanrovi (zanr varchar(20));
INSERT INTO zanrovi VALUES ('Dramas'), ('Comedies'), ('Documentaries'), ('Action & Adventure');

SELECT * FROM crosstab(
    $$SELECT duration::time, genrename, COUNT(DISTINCT ownerid)::int
    FROM track NATURAL JOIN trackview NATURAL JOIN trackgenre NATURAL JOIN genre
    WHERE genrename IN ('Dramas', 'Comedies', 'Documentaries', 'Action & Adventure')
    AND to_tsvector('english',tracktitle || description) @@ to_tsquery('english','activ|worthi|ride')
    GROUP BY duration, genrename ORDER BY duration, genrename$$,
    $$SELECT zanr FROM zanrovi ORDER BY zanr$$)
AS pivottable(duration time, action_adventure int, comedies int, documentaries int, dramas int)
```

ZADATAK 5

Za filmove koji u **nazivu** sadrže riječ nalik riječi **'manster'** ispisati naziv filma te naziv **druge** kategorije kojoj film pripada (*categories*).

Smatrajte da riječ iz naslova filma nalikuje riječi **'manster'** ako je potrebno obaviti manje od 3 promjene da bi riječi postale jednake.

Nije potrebno ispisati podatke za filmove za koje nije navedena (ne postoji) **druga** kategorija.

Donja tablica ilustrira izgled rezultata.

```
SELECT tracktitle, categories[2] AS secondcategory
FROM track,
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)
WHERE categories[2] IS NOT null
AND levenshtein(word, 'manster') < 3
```

ZADATAK 6

Ispisati ograničenja na sadržaje ovisno o starosnoj dobi (*tAgeRestriction*) **filmova** i ukupan broj profila koji su gledali/pokretali te filmove. U obzir uzeti samo filmove koji:

- **bilo u nazivu bilo u opisu** sadrže riječ čiji je normalizirani oblik jednak **'activ'** ili **'ride'**
- imaju zvučni zapis na nekom od jezika: 'English', 'German', 'Indonesian', 'Korean'.

Donja tablica ilustrira izgled rezultata.

tagerestriction	'english'	'German'	'indonesian'	'korean'
...	...	...	...	...
14	5	3	null	null

```
CREATE TEMP TABLE titlovi(titl varchar(15));
INSERT INTO titlovi VALUES ('English'), ('German'), ('Indonesian'), ('Korean');

SELECT * FROM crosstab(
    $$SELECT tagerestriction::int as tar, langname, COUNT (DISTINCT ownerid)::int
    FROM track NATURAL JOIN trackview NATURAL JOIN audiolang JOIN language l ON audiolang.audiolangid = l.langid
    WHERE to_tsvector('english', tracktitle || description) @@ to_tsquery('english', 'activ|ride')
    AND langname IN ('English', 'German', 'Indonesian', 'Korean')
    GROUP BY tar, langname ORDER BY tar, langname$$,
    $$SELECT titl FROM titlovi ORDER BY titl$$
)
AS pivotTable(tagerestriction int, english int, German int, indonesian int, korean int)
```

ZADATAK 7

Ispisati identifikator i naziv žanra te broj **filmova** koji pripadaju tom žanru. Pored **filmova** koji žanru pripadaju temeljem evidencije u *trackGenre* smatra se da mu pripadaju i **filmovi** koji pripadaju nekom od njemu podređenih žanrova (direktno i indirektno podređeni). Nije potrebno ispisati podatke za žanrove kojima ne pripada niti jedan **film**.

Donja tablica ilustrira izgled rezultata.

```
WITH RECURSIVE R(genreid, genrename, supgenreid, trackid) AS (
    SELECT genre.genreid, genrename, supgenreid, movie.trackid
    FROM movie NATURAL JOIN trackgenre NATURAL JOIN genre
    UNION
    SELECT genre.genreid, genre.genrename, genre.supgenreid, R.trackid
    FROM R, genre WHERE R.supgenreid = genre.genreid
)
SELECT genreid, genrename, COUNT(trackid) AS numofmovies
FROM R
GROUP BY genreid, genrename
```

ZADATAK 8

Za filmove čiji **naziv** sadrži riječ nalik riječi **'colect'** ispisati naziv filma te ime i prezime prvog režisera. Nalikuje li riječ iz naslova filma riječi **'colect'** ispitajte pomoću implementacije Q-gram algoritma u PostgreSQL-u.

```
SELECT DISTINCT tracktitle, director[1] AS firstdirector
FROM movie NATURAL JOIN track,
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)
WHERE similarity(word, 'colect') > 0.3
```

ZADATAK 9

Ispisati prezimena i imena vlasnika korisničkih računa (u obliku kao u tablici dolje) i ukupan broj poznatih ocjena (liked) za pokrenuti medijski sadržaj od strane bilo kojeg profila koji pripada tom korisničkom računu (bez obzira radi li se o filmu ili epizodi neke serije).

U obzir uzeti samo:

- sadržaje koji pripadaju nekom od žanrova: 'Thrillers', 'Stand-Up Comedy', 'Family Movies', 'Sports Movies', 'Horror'
- vlasnike čije je ime prema metodi koja sličnost utvrđuje temeljem broja podudarnih trigrama slično barem 0.2 imenu 'Leon'.

```
CREATE TEMP TABLE zanrovi(zanr varchar(20));
INSERT INTO zanrovi VALUES ('Thrillers'), ('Stand-Up Comedy'), ('Family Movies'), ('Sports Movies'), ('Horror');

SELECT * FROM crosstab(
    $$SELECT lastname||' '||firstname::varchar as ownername, genrename, count(liked)
    FROM owner NATURAL JOIN profiletrack NATURAL JOIN track NATURAL JOIN trackgenre NATURAL JOIN genre
    WHERE genrename IN ('Thrillers', 'Stand-Up Comedy', 'Family Movies', 'Sports Movies', 'Horror')
    AND similarity(firstname, 'Leon') >= 0.2 --AND profiletrack.liked IS NOT NULL
    GROUP BY ownername, genrename ORDER BY ownername, genrename$$,
    $$SELECT zanr FROM zanrovi ORDER BY zanr$$)
AS pivotTable(ownername varchar, family_movies int, horror int, sports_movies int, stand_up_comedy int, thrillers int)
```

ZADATAK 10

Ispisati naslov epizode serije, vrijeme njenog pokretanja u obliku **hh:mm:ss** (od strane bilo kojeg profila bilo kojeg korisničkog računa, bilo kojeg datuma) i broj pokretanja te iste epizode u vremenskom intervalu koji obuhvaća to vrijeme i prethodna 4 sata i to bez obzira na datum.

U obzir uzeti samo epizode koje u nazivu sadrže riječ koja će u ne više od 3 izmjene postati jednaka riječi **'vidaut'**.

Pri usporedbi riječi nemojte praviti razliku između velikih i malih slova.

```
SELECT tracktitle, viewstartdatetime::time,
SUM(COUNT(DISTINCT ownerid)) OVER (PARTITION BY tracktitle ORDER BY trackview.viewstartdatetime::TIME::INTERVAL
                                   RANGE BETWEEN '4 hour'::interval PRECEDING AND CURRENT ROW)
FROM showep NATURAL JOIN track NATURAL JOIN trackview,
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)
WHERE levenshtein(lower(word), 'vidaut') <= 3
GROUP BY tracktitle, viewstartdatetime
```

ZADATAK 11

Ispisati naslov **filma**, datum njegovog pokretanja (od strane bilo kojeg profila bilo kojeg korisničkog računa) i broj pokretanja tog istog filma u vremenskom intervalu koji obuhvaća taj datum i sljedećih 6 mjeseci.

U obzir uzeti samo filmove koje u nazivu sadrže riječ koja će u ne više od 2 izmjene postati jednaka riječi **'tings'** i pri tom ne pravite razliku između velikih i malih slova (tings, Tings, TINGS - sve riječi smatrajte jednakim).

Donja tablica ilustrira izgled rezultata.

```
SELECT distinct tracktitle, viewstartdatetime::date as viewstartdate,
COUNT(viewstartdatetime) OVER (PARTITION BY tracktitle ORDER BY viewstartdatetime::date
                                RANGE BETWEEN CURRENT ROW AND '6 month'::interval FOLLOWING) AS currentAndNext6Mon
FROM movie NATURAL JOIN track NATURAL JOIN trackview NATURAL JOIN profiletrack,
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)
WHERE levenshtein(lower(word), 'tings') <= 2
GROUP BY tracktitle, viewstartdate, viewstartdatetime
ORDER BY tracktitle, viewstartdate
```

ZADATAK 12

Ispisati različite mjesece izdanja **filmova** i ukupan broj poznatih ocjena (liked) za film bez obzira jesu li pozitivne ili negativne.

U obzir uzeti samo filmove koji:

- koji **bilo u nazivu bilo u opisu** sadrže riječ čiji je normalizirani oblik jednak **'ugli'** ili **'worthi'** ili **'ride'**
- pripadaju nekom od žanrova: 'Thrillers', 'Stand-Up Comedy', 'Family Movies', 'Sports Movies', 'Horror'.

Donja tablica ilustrira izgled rezultata.

```
CREATE TEMP TABLE zanrovi(zanr varchar(20));
INSERT INTO zanrovi VALUES ('Thrillers'), ('Stand-Up Comedy'), ('Family Movies'), ('Sports Movies'), ('Horror');

SELECT * FROM crosstab(
    $$SELECT EXTRACT(MONTH FROM releasedate) AS releasemonth, genrename, count(liked)
    FROM movie NATURAL JOIN track NATURAL JOIN trackgenre NATURAL JOIN genre NATURAL JOIN profiletrack
    WHERE genrename IN ('Thrillers', 'Stand-Up Comedy', 'Family Movies', 'Sports Movies', 'Horror')
    AND to_tsvector('english', tracktitle||' ||description) @@ to_tsquery('english', 'ugli|worthi|ride')
    AND liked IS NOT NULL
    GROUP BY releasemonth, genrename ORDER BY releasemonth, genrename$$,
    $$SELECT zanr FROM zanrovi ORDER BY zanr$$
)
AS pivotTable(genrename int, family_movies int, horror int, sports_movies int, stand_up_comedy int, thrillers int)
```

ZADATAK 13

Za filmove koji u **nazivu** sadrže riječ nalik riječi '**maus**' ispisati naziv filma te ime i prezime osobe koja je navedena kao **peta** od svih koje su sudjelovale u stvaranju filma (*castAndCrew*). Smatrajte da riječ iz naslova filma nalikuje riječi '**maus**' ako je potrebno obaviti manje od 3 promjene da bi riječi postale jednake.

Donja tablica ilustrira izgled rezultata.

```
SELECT DISTINCT tracktitle, castandcrew[5] AS fifthcastandcrew
FROM movie NATURAL JOIN track,
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)
WHERE castandcrew[5] IS NOT null
AND levenshtein(word, 'maus') < 3
```

ZADATAK 14

Po žanrovima i godinama ispisati ukupan broj pregleda/pokretanja **filmova** koji pripadaju tom žanru. U obzir uzeti godine od 2016 do 2021.

Pored **filmova** koji žanru pripadaju temeljem evidencije u *trackGenre* smatra se da mu pripadaju i **filmovi** koji pripadaju nekom od njemu podređenih žanrova (direktno i indirektno podređeni). Nije potrebno ispisati podatke za žanrove kojima ne pripada niti jedan film (niti direktno niti indirektno preko podređenih žanrova).

genrename	y2016	y2017	y2018	y2019	y2020	y2021
...	...	...	...	...	...	...
Music & Musicals	79	74	112	126	195	74

```
CREATE TEMP TABLE godine(god int);
INSERT INTO godine VALUES (2016), (2017), (2018), (2019), (2020), (2021);

CREATE TEMP TABLE RR AS
WITH RECURSIVE R(genrename, genreid, supgenreid, viewstartdatetime) AS (
    SELECT genrename, genreid, supgenreid, viewstartdatetime
    FROM movie NATURAL JOIN track NATURAL JOIN trackgenre NATURAL JOIN genre NATURAL JOIN trackview
    UNION ALL
    SELECT genre.genrename, genre.genreid, genre.supgenreid, R.viewstartdatetime
    FROM R, genre
    WHERE genre.genreid = R.supgenreid
) SELECT * FROM R;

SELECT * FROM crosstab(
    $$SELECT genrename, EXTRACT(YEAR FROM viewstartdatetime) AS myear, COUNT(viewstartdatetime)
    FROM RR WHERE EXTRACT (YEAR FROM viewstartdatetime) IN (2016, 2017, 2018, 2019, 2020, 2021)
    GROUP BY genrename, myear ORDER BY genrename, myear$$,
    $$SELECT god FROM godine ORDER BY god$$
) AS pivotTable(genrename varchar, y2016 int, y2017 int, y2018 int, y2019 int, y2020 int, y2021 int)
```

ZADATAK 15

Ispisati različite godine izdanja **filmova** i ukupan broj pozitivnih ocjena (liked) za film. U obzir uzeti samo filmove koji:

- **bilo u nazivu bilo u opisu** sadrže riječ čiji je normalizirani oblik jednak normaliziranom obliku 'ugli' ili 'worthi' ili 'famili'
- pripadaju nekom od žanrova: 'Thrillers', 'Family Movies', 'Stand-Up Comedy', 'Horror', 'Sports Movies', 'Music & Musicals'

Donja tablica ilustrira izgled rezultata.

releaseyear	family_movies	horror	music_musicals	sports_movies	stand_up_comedy	thrillers
...	...	...	...	...	...	...
2013	3	null	null	null	null	null

```
CREATE TEMP TABLE zanrovi(zanr varchar(20));
INSERT INTO zanrovi VALUES ('Thrillers'), ('Family Movies'), ('Stand-Up Comedy'), ('Horror'), ('Sports Movies'), ('Music & Musicals');

SELECT * FROM crosstab(
    $$SELECT EXTRACT(YEAR FROM releasedate) AS releaseyear, genrename, COUNT(liked)
    FROM movie NATURAL JOIN track NATURAL JOIN trackgenre NATURAL JOIN genre NATURAL JOIN profiletrack
    WHERE genrename IN ('Thrillers', 'Family Movies', 'Stand-Up Comedy', 'Horror', 'Sports Movies', 'Music & Musicals')
    AND to_tsvector('english', tracktitle || ' ||description') @@ to_tsquery('english', 'ugli|worthi|famili')
    AND liked > 0
    GROUP BY releaseyear, genrename ORDER BY releaseyear, genrename$$,
    $$SELECT zanr FROM zanrovi ORDER BY zanr$$)
AS pivotTable(releaseyear int, family_movies int, horror int, music_musicals int, sports_movies int, stand_up_comedy int, thrillers int)
```

ZADATAK 16

Ispisati identifikator i naziv žanra te broj podređenih žanrova (direktno i indirektno podređenih). Smatrajte da žanr nije sam sebi niti nadređen niti podređen pa je za žanrove koji nemaju podređenih potrebno ispisati vrijednost 0 za broj podređenih.

```
WITH RECURSIVE R(genreid, genrename, supgenreid) AS (
    SELECT genreid, genrename, supgenreid
    FROM genre
    UNION
    SELECT R.genreid, R.genrename, genre.supgenreid
    FROM R, genre
    WHERE genre.genreid = R.supgenreid
)
SELECT DISTINCT genreid, genrename, (SELECT COUNT(*) FROM R R2 WHERE R.genreid = R2.supgenreid) AS numofsub
FROM R
GROUP BY genreid, genrename
```

ZADATAK 17

Ispisati identifikator i naziv žanra te ukupan broj pozitivnih ocjena (liked) za **filmmove** koji pripadaju tom žanru.

Pored **filmova** koji žanru pripadaju temeljem evidencije u *trackGenre* smatra se da mu pripadaju i **filmovi** koji pripadaju nekom od njemu podređenih žanrova (direktno i indirektno podređeni).

Nije potrebno ispisati podatke za žanrove kojima ne pripada niti jedan film (niti direktno niti indirektno preko podređenih žanrova).

```
WITH RECURSIVE R(genreid, genrename, supgenreid, liked) AS (  
  SELECT genreid, genrename, supgenreid, liked  
  FROM genre NATURAL JOIN trackgenre NATURAL JOIN movie NATURAL JOIN profiletrack  
  UNION ALL  
  SELECT genre.genreid, genre.genrename, genre.supgenreid, R.liked  
  FROM R, genre  
  WHERE genre.genreid = R.supgenreid  
)  
SELECT DISTINCT genreid, genrename, COUNT(liked) AS numoflikes  
FROM R WHERE liked = 1  
GROUP BY genreid, genrename
```

ZADATAK 18

Ispisati ime i prezime vlasnika korisničkog računa, godinu (vremenskog trenutka *trackView.viewStartDateTime*) i broj **epizoda serija** pokrenutih od strane svih profila korisničkog računa u toj godini.

Dodatno ispisati kumulativni broj pokretanja (*cntThisAndPrevYear*), koji obuhvaća i pokretanja iz svih prethodnih godina.

U obzir uzeti samo serije koje u naslovu sadrže riječ koja će uz najviše 2 promjene biti jednaka riječi **'pauver'**.

```
SELECT firstname, lastname, EXTRACT(YEAR FROM viewstartdatetime) as date_part, COUNT(trackview.ownerid) AS cntinyear,  
SUM(COUNT(viewstartdatetime)) OVER (PARTITION BY trackview.ownerid ORDER BY EXTRACT(YEAR FROM viewstartdatetime)  
                                     RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cntthisandprevyears  
FROM trackview NATURAL JOIN owner NATURAL JOIN track NATURAL JOIN showep,  
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)  
WHERE levenshtein(lower(word), 'pauver') <= 2  
GROUP BY firstname, lastname, date_part, trackview.ownerid
```

ZADATAK 19

Ispisati naslov serije (show), njezinu ocjenu/rating, broj pokretanja svih epizoda serije i rang izračunat temeljem:

- broja pokretanja svih epizoda (*rangCnt*: najviše pokretanja - najbolji rang)
- ocjene/rejtinga serije (*rangRating*: najbolja ocjena-najbolji rang)

U obzir uzeti samo serije koje u nazivu sadrže riječ koja će u ne više od 2 izmjene postati jednaka riječi **'sikret'**.

Pri usporedbi riječi nemojte praviti razliku između velikih i malih slova.

```
SELECT showtitle, showrating, COUNT(viewstartdatetime),
DENSE_RANK() OVER (ORDER BY COUNT(viewstartdatetime) DESC) AS rangcnt,
DENSE_RANK() OVER (ORDER BY showrating DESC) AS rangrating
FROM show NATURAL JOIN showep NATURAL JOIN track NATURAL JOIN trackview,
UNNEST(string_to_array(tracktitle, ' ')) AS words(word)
WHERE levenshtein(lower(word), 'sikret') <= 2
GROUP BY showtitle, showrating
```

ZADATAK 20

Ispisati ime i prezime osoba koje su sudjelovale u nastajanju medijskog sadržaja (atribut *castAndCrew*) i ukupan broj medijskih sadržaja u kojima je osoba sudjelovala.

Ispisati i rang osobe tako da su najbolje rangirane one koje su sudjelovale u najviše medijskih sadržaja. Ispisati podatke samo za osobe čije ime počinje nizom znakova **'Kate'**.

Well 'Kate% ', s razmakom, jer je netko failao u postavljanju zadatka, moguće da je trebalo biti sve osobe imena Kate

```
SELECT cc.person AS firstandlastname, COUNT(trackid) AS nooftracks,
RANK() OVER (ORDER BY COUNT(trackid) DESC) AS rang
FROM track,
UNNEST(castandcrew) AS cc(person)
WHERE person LIKE 'Kate%'
GROUP BY firstandlastname
```

ZADATAK 21

GIS

ZADATAK 1

Pretpostavite da želite za sve vatrogasne postaje (buildings) naći 2 najbliže rijeke (water, "river") i udaljenost do njih.

Za svaku zgradu koja u nazivu sadrži niz znakova "vatrog" ili "dvd" (bez obzira na velika ili mala slova) ispisati: ime zgrade, ime rijeke, udaljenost do rijeke i redni broj.

Rezultate poredajte po imenu zgrade i rednom broju rijeke.

Udaljenosti zaokružite na cijeli broj.

```
SELECT * FROM (  
SELECT buildings.name AS depname, water.name AS rname, ROUND(ST_Distance(buildings.geom, water.geom)) AS dist,  
RANK() OVER (PARTITION BY buildings.name ORDER BY ST_Distance(buildings.geom, water.geom) ASC) AS rn  
FROM buildings, water  
WHERE (LOWER(buildings.name) LIKE '%dvd%' OR LOWER(buildings.name) LIKE '%vatrog%')  
AND water.fclass = 'river'  
ORDER BY depname ASC, rn) R  
WHERE rn IN (1,2)
```

ZADATAK 2

Pronaći pet mjesta (places) koja su najviše izolirana (najviše udaljena od drugih mjesta).

Potrebno je ispisati gid, naziv mjesta i udaljenost do najbližeg mjesta.

Zapise poredati silazno s obzirom na udaljenost do najbližeg mjesta.

```
SELECT p1.gid, p1.name, MIN(ST_Distance(p1.geom, p2.geom)) AS dist  
FROM places p1, places p2  
WHERE p1.gid != p2.gid  
GROUP BY p1.gid, p1.name  
ORDER BY dist DESC  
LIMIT 5
```

ZADATAK 3

Za svaku šumu (landuse, "forest") kojoj je definirano ime i zgradu koja u nazivu sadrži niz znakova "vatrog" ili "dvd" (bez obzira na velika ili mala slova) ispisati: ime šume, ime zgrade, udaljenost do zgrade i redni broj.

Rezultate poredajte po imenu šume i rednom broju vatrogasne postaje.

Udaljenosti zaokružite na cijeli broj.

```
SELECT * FROM (  
SELECT landuse.name fname, buildings.name depname, ROUND(ST_Distance(landuse.geom, buildings.geom)) AS dist,  
RANK() OVER (PARTITION BY landuse.name ORDER BY ST_Distance(landuse.geom, buildings.geom) ASC) AS rn  
FROM landuse, buildings  
WHERE landuse.fclass = 'forest' AND landuse.name IS NOT null --AND buildings.fclass = 'apartment'  
AND (lower(buildings.name) LIKE '%dvd%' OR lower(buildings.name) LIKE '%vatrog%')  
ORDER BY fname ASC, rn) R  
WHERE rn IN (1,2,3)
```

ZADATAK 4

Ispišite imena i ukupnu duljinu cesta koje u nazivu sadržavaju riječ 'šetalište' (bez obzira na velika ili mala slova) i koja nekim svojim dijelom prolaze kroz šumu. Duljinu zaokružite na cijeli broj.

Idk, close enough

```
SELECT DISTINCT roads.name, SUM((ST_Length(roads.geom))::int
FROM roads, landuse
WHERE LOWER(roads.name) LIKE '%šetalište%' AND landuse.fclass = 'forest'
AND ST_Intersects(roads.geom, landuse.geom)
GROUP BY roads.name
```

ZI 2021/2022

ZADATAK 1

2PC protokol

1. Što je **2PC** protokol i čemu služi?
 2. Kod kojih sustava se koristi?
 3. Koja su njegova svojstva?
-
1. protokol koji osigurava atomarnost globalne transakcije (ili su sve subtransakcije obavljene ili nijedna)
 2. u distribuiranim bazama podataka
 3. 2PC protokol je blokirajući protokol, nije nezavisan s obzirom na mogućnost obnove

ZADATAK 2

Pomoću kolekcije `nobelprizes` doznajte kategorije s najviše laureata za svaku dekadu u intervalu [1900, 2000).

Kategorije staviti u polje `top` koje treba biti abecedno sortirano (u JS-u možete sortirati polje tako da jednostavno pozovete metodu `.sort()` koja će *in-place* sortirati polje).

Dodatno, dohvatiti i minimalni i maksimalni broj laureata po dekadama.

Za dekadu:

- [1900, 1909] koristiti oznaku "1900"

Primjer zapisa:

```
{
  "_id": 1960,
  "value": {
    "min": 2,
    "max": 25,
    "top": [
      "medicine" // medicine was a top category with 25 Laureats
    ]
  }
}
```

```
db.nobelprizes.mapReduce(
  `function() {
    var self = this;
    if (this.year >=1900 && this.year < 2000) {
      var decade = this.year - this.year % 10;
      let value = {};
      value[this.category] = this.laureates.length;
      emit(decade, value);
    }
  }`,
  `function(key, values) {
    var rv = {};
    values.forEach( function(value) {
      Object.keys(value).forEach(key => {
        if (rv[key]) {
          rv[key] += value[key];
        } else {
          rv[key] = value[key];
        }
      });
    });
    return rv;
  }`,
```

```
{
  finalize: `
function (key, value) {
  let sum = 0, cnt = 0, min = 100000, max = 0;
  Object.keys(value).forEach(key => {
    if (value[key] > max) max = value[key];
    if (value[key] < min) min = value[key];
  });
  let top = [];
  Object.keys(value).forEach(key => {
    if (value[key] == max) top.push(key);
  });
  top.sort();
  value.top = top;
  return {
    min, max, top
  }
}
`
}
```

ZADATAK 3

Objasnite zašto se, u svijetu NoSQL tehnologija, smatra da grafovske baze podataka odudaraju od ostalih ("Strange fish in SQL pond")?

- Razgrađuje podatke u još manje jedinice od RBP-a
- Nema agregatnog modela
- Nisu pogodne za distribuciju
- Sličnije relacijskim bazama, podržavaju transakcije (ACID)

Navedite i objasnite po stavkama što je grafovskim bazama podataka zajedničko, a što nije, s ostalim NoSQL tehnologijama.

- Zajedničko s ostalima: ne-relacijski model, popularnost, open source
- Što nije zajedničko - Pogodni za složene, polu-strukturirane, jako povezane podatke
- Navedite i objasnite kategorije konzistencije u sustavima tokova podataka.

ZADATAK 4

1. Tri kategorije konzistencije (consistency, correctness, accuracy,...):

1. At-most-once-processing
2. At-least-once-processing
3. Exactly-once-processing/effectively-once processing
 - To ne znači da će sustav samo jednom (pokušati) obraditi događaj, već da će na izlaz biti proslijeđena samo jedna od tih obrada

2. Koju je najteže ostvariti?

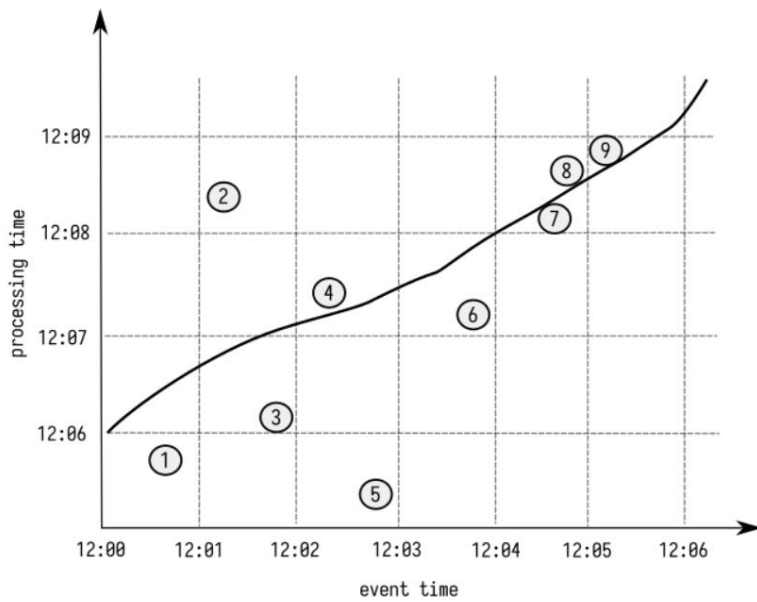
Exactly-once-processing/effectively-once processing

3. Kako je ta konzistencija povezana s izvorištem i odredištem tokova?

Treba ostvariti exactly-once i u interakciji s izvorištima i odredištima

ZADATAK 5

Na slici je prikazan slijed događaja u *event/processing time* domeni i heuristički *watermark*, pri čemu je vrijednost događaja koja nas zanima označena u krugu i radi jednostavnosti poprima vrijednosti 1-9:



```
PCollection<KV<Team, Integer>> totals = input
    .apply(Window.into(FixedWindows.of(TWO_MINUTES))
        .triggering(
            AfterWatermark()
            .withEarlyFirings(AlignedDelay(TWO_MINUTES))
            .withLateFirings(AfterCount(2)))
        .discardingFiredPanels())
    .apply(Sum.integersPerKey());
```

```
(12:07, 12:00-12:02, E, 4)
(12:07, 12:02-12:04, E, 5)
(12:08, 12:02-12:04, O, 10)
(12:09, 12:04-12:06, E, 24)
```

ZADATAK 7

Objasnite ulogu i osobine činjeničnih tablica te ulogu i osobine dimenzijskih tablica u zvjezdastom dimenzijskom modelu podataka.

Usporedite zauzeće prostora (memorije) od strane činjeničnih i dimenzijskih tablica

- za jednu ntorku
- po pitanju ukupnog broja ntorki u svakoj

Činjenična tablica odgovara procesu koji se prati u skladištu podataka. Sadrži dvije skupine brojčanih atributa:

- ključevi dimenzijskih tablica
- mjere (engl. measures)
Često je (gotovo) identična odgovarajućoj tablici u relacijskom izvoru podataka - normalizirane je sheme.

Dimenzijska tablica predstavlja subjekt/objekt koji sudjeluje u procesu koji se prati u skladištu podataka.

Objašnjavaju činjenice pohranjene u činjeničnoj tablici, odnosno opisuju kontekst, npr.: koji student ja kada položio predmet kod kojeg nastavnika.

Dimenzijska tablica nije normalizirana.

Često nastaje spajanjem više relacijskih tablica.

Zauzeće memorije:

Činjenične tablice: velik broj zapisa (10^5 , 10^6 , 10^7 , ...) jedan redak (n-torka) ne zauzima puno prostora (normalizirana tablica, numerički atributi)

Dimenzijske tablice: mali broj zapisa ($<10^5$, većinom $<10^2$) veliko zauzeće prostora po jednom retku (nenormalizirana, npr. 30 atributa, od toga 15-tak znakovni nizovi: $15 * 4 + 15 * 100 = 1560$ byte)

ZADATAK 1

Na slici je prikazan RDF graf za segment TBoxa i ABoxa s DBPedia kojim se definira

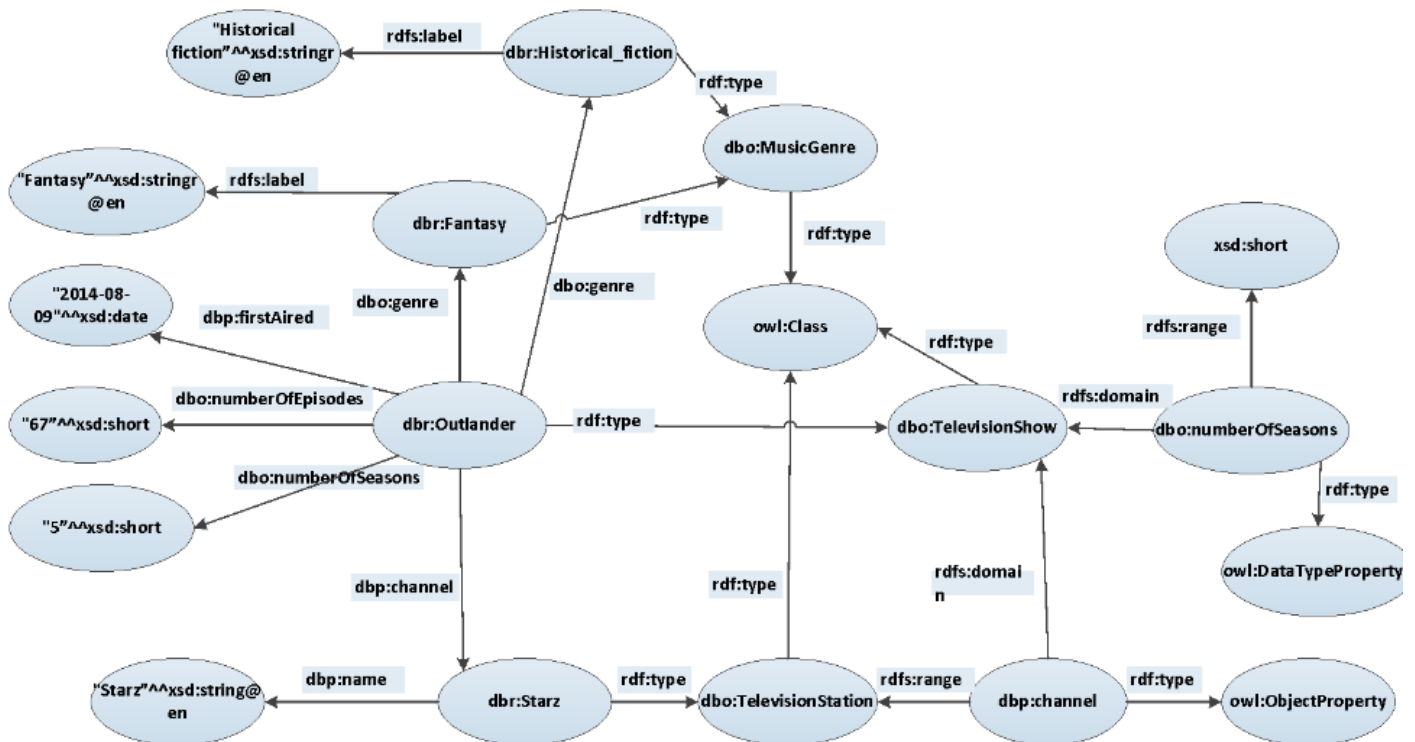
- terminološko znanje o TV serijama, žanrovima kojima pripadaju i TV mrežama (npr. Starz)
- asertivno znanje o popularnoj TV seriji Outlander (služi samo za ilustraciju)

Napišite SPARQL upit kojim će biti ispisan naziv TV mreže, naziv žanra na engleskom jeziku i broj njenih TV serija u pripadnom žanru.

U obzir uzeti samo TV serije s više od 3 sezone i više od 20 epizoda ukupno. Rezultat poredati silazno prema broju TV serija.

Smatrati da

- se upit izvodi na dbpedijinom SPARQL endpointu <http://dbpedia.org/sparql>.
- se koristi defaultni RDF graf <http://dbpedia.org>
- su predefinjirani svi prostori imena koji su inače predefinjirani na dbpedijinom SPARQL endpointu.



#PREFIX opcionalno jer su svi predefinjirani na dbpedia sparql endpointu
 SELECT ?TVStationName ?genreLabel COUNT(?TVShowURI) AS ?noOfShows
 #FROM <http://dbpedia.org> - opcionalno jer je ovaj defaultni

```
WHERE {
  ?TVShowURI      a          dbo:TelevisionShow;
                  dbp:channel ?TVStationUri ;
                  dbo:numberOfSeasons ?noOfSeasons ;
                  dbo:numberOfEpisodes ?noOfEpisodes ;
                  dbo:genre      ?genreURI .
  ?genreURI       rdfs:label  ?genreLabel .
  ?TVStationUri   a          dbo:TelevisionStation;
                  dbp:name     ?TVStationName .
  FILTER (?noOfSeasons > 3 && ?noOfEpisodes > 20)
  FILTER (LANG(?genreLabel)="en")
}
```

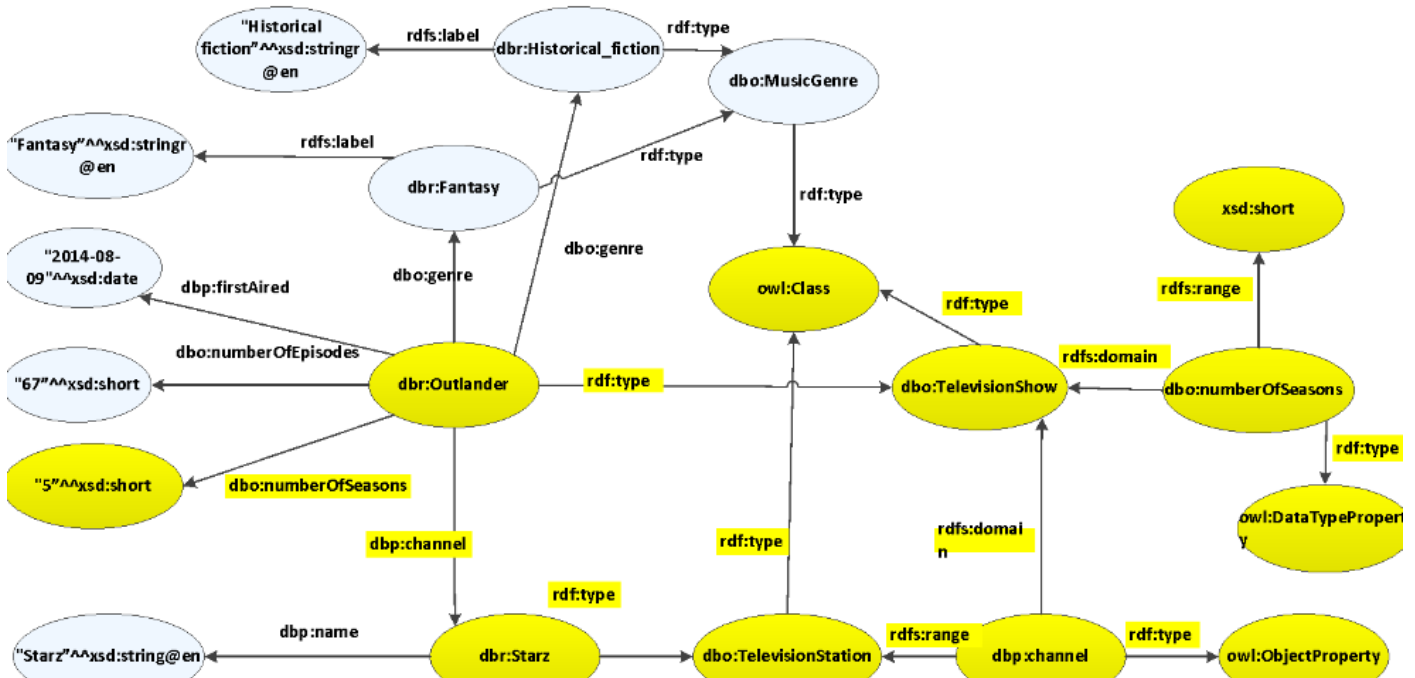
GROUP BY ?TVStationName?genreLabel
 ORDER BY DESC (?noOfShows)

ZADATAK 2

Na slici je prikazan RDF graf za segment TBoxa i ABoxa s DBPedia.

Napišite Turtle izraze za segment grafa obojan žutom bojom.

Napomena: Prefikse svih prostora imena koji su navedeni u službenom podsjetniku možete koristiti i bez prepisivanja definicija prefiksa.



```

dbo:TelevisionStation a owl:Class
dbo:TelevisionShow    a owl:Class

dbp:channel a          owl:ObjectProperty ;
               rdfs:domain dbo:TelevisionShow ;
               rdfs:range  dbo:TelevisionStation .

dbo:numberOfSeasons a      owl:DatatypeProperty ;
               rdfs:domain dbo:TelevisionShow ;
               rdfs:range  xsd:Short.

dbr:Starz a dbo:TelevisionStation ;

dbr:Outlander a          dbo:TelevisionShow ;
               dbp:channel      dbr:Starz ;
               dbo:numberOfSeasons "5"^^xsd:short ;

```

ZADATAK 3

Pomoću kolekcije `nobelprizes` doznajte broj laureata po kategorijama za svaku dekadu u intervalu [1900, 2000). Dodatno, dohvatiti i prosječan broj laureata po dekadama. Za dekadu:

- [1900, 1909] koristiti oznaku "1900"

```
{
  "_id": "chemistry",
  "value": {
    "1900": 9,
    "1910": 8,
    "1920": 10,
    ...
    "1990": 18,
    "avg": 13.2
  }
}
```

```
db.nobelprizes.mapReduce(
  `function() {
    var self = this;
    if (this.year >= 1900 && this.year < 2000) {
      var decade = this.year - this.year % 10;
      let value = {};
      value[decade] = this.laureates.length;
      emit(this.category, value);
    }
  }`,
  `function(key, values) {
    var rv = {};
    values.forEach( function(value) {
      Object.keys(value).forEach(key => {
        if (rv[key]) {
          rv[key] += value[key];
        } else {
          rv[key] = value[key];
        }
      });
    });
    return rv;
  }`,
  finalize: `
function (key, value) {
  let sum = 0, cnt = 0;
  Object.keys(value).forEach(key => {
    sum += value[key];
    ++cnt;
  });
  value.avg = sum / cnt;
  return value;
}
`);
```

```
function (key, value) {
  let sum = 0, cnt = 0;
  Object.keys(value).forEach(key => {
    sum += value[key];
    ++cnt;
  });
  value.avg = sum / cnt;
  return value;
}
`);
```

ZADATAK 4

2PL protokol

1. Što je **2PL** protokol i čemu služi?
 2. Kod kojih sustava se koristi?
 3. Koja su njegova svojstva?
 4. Postoji li alternativa tom protokolu i koja?
-
1. Protokol zasnovan na zaključavanju za osiguravanje **serijalizabilnosti** transakcija u bazama podataka. Ima dvije faze: fazu rasta (postavljanje ključeva) i fazu sužavanja (otpuštanje ključeva)
 2. Koristi se primarno u relacijskim bazama podataka, ali mogao bi se koristiti i šire.
 3. Pesimističan, koristi zaključavanje, loše performanse
 4. Drugi protokoli za serijalizabilnost transakcija, poput PostgreSQL-ovog SSI zasnovanog na MVCC-u ili trivijalnog serijskog obavljanja transakcija kao u VoltDB

ZADATAK 5

Fragmentacija (*sharding*)

a) (1 bod)

Objasnite problem preraspodjele fragmenata u repliciranom sustavu; kojima načelima se treba voditi prilikom preraspodjela?

b) (3 boda)

Navedite i objasnite strategije preraspodjele fragmenata.

// ovo pitanje nije o tome kako napraviti sharding, tj. hash based i range based i sl. to se **NE traži**

(a) Vremenom opterećenje (količina podataka) raste te rastu veličina odnosno broj fragmenata i potrebno je napraviti preraspodjelu fragmenata, pri čemu se očekuje:

- Baza mora biti operativna za vrijeme preraspodjele
- Pravedna raspodjela opterećenja nakon preraspodjele
- Minimizirati mrežni promet prilikom preraspodjele Za to je potrebno koristiti neku vrstu consistent hashinga

(b) Postoje sljedeće strategije (vidjeti NoSQL4 S22-):

1. Fiksni broj fragmenata koji je napravljeni unaprijed, (npr. Riak ring)
 - Novi čvor preuzima svoj udio cijelih fragmenata od postojećih čvorova
2. Dinamički broj fragmenata (npr. Mongo)
 - Svaki fragment pripada jednom čvoru
 - Ako fragment naraste preko definirane veličine
 - Razdijeliti na dva jednaka dijela
 - Potencijalno jedan fragment prebaciti na drugi čvor
3. Fiksni broj fragmenata po čvoru (npr. Cassandra)
 - Kada se novi čvor priključi, odabire slučajno postojeći čvor i podijeli s njim opterećenje
 - Prije: postojeći čvor s N fragmenata, ukupno X MB
 - Poslije:
 - postojeći čvor s N fragmenata, ukupno ~ X/2 MB
 - novi čvor s N fragmenata, ukupno ~ X/2 MB

ZADATAK 6

Tokovi podataka:

a) (2 boda)

Koje pristupe prijenosu događaja preko mreže znate? Komentirajte njihova svojstva i razlike, navedite primjer sustava ako znate.

a) (1 boda)

Općenito, što se može dogoditi kada proizvođači šalju poruke brže nego što ih potrošači mogu primiti? Komentirajte opcije.

(a) Dva pristupa:

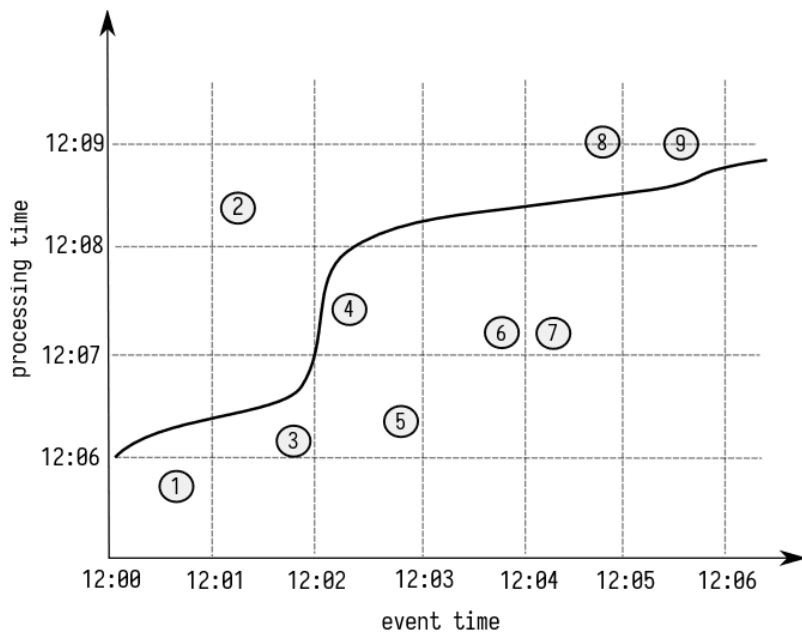
1. **Direktna komunikacija** od proizvođača do potrošača - moguć gubitak poruka, potrebno je napraviti vlastito aplikacijsko rješenje da se to spriječi ako se želi - npr. ZeroMQ
2. **Posrednički sustavi** - Posrednik je poslužitelj, a proizvođači i potrošači klijenti - Tipično asinkrona komunikacija, proizvođač samo preda poruku posredniku, koja naknadno biva dostavljena potrošaču - Ovdje imaju dva podpristupa:
 1. Message Broker/Queue
 - postoje i dva standarda: AMQP, JMS
 - npr. RabbitMQ
 2. Log-based
 - Proizvođači dodaju u dnevnik
 - Potrošači čitaju (ali ne brišu)
 - Radi performansi - dnevnik se razlaže u particije (i logički - teme)

(b) Moguće je:

- Odbaciti poruke
- Spremati poruke u red (dokad?)
- Kočiti proizvođače (backpressure, flow control), npr. unix pipes
- (kombinacija pristupa)

ZADATAK 7

Na slici je prikazan slijed događaja u *event/processing time* domeni i heuristički *watermark*, pri čemu je vrijednost događaja koja nas zanima označena u krugu i radi jednostavnosti poprima vrijednosti 1-9



```
PCollection<KV<Team, Integer>> totals = input
    .apply(Window.into(FixedWindows.of(TWO_MINUTES))
        .triggering(
            AfterWatermark()
            .withEarlyFirings(AlignedDelay(ONE_MINUTE))
            .withLateFirings(AfterCount(2)))
        .discardingFiredPanels())
    .apply(Sum.integersPerKey());
```

```
(12:06, 12:00-12:02, E, 1)
(12:07, 12:00-12:02, 0, 3)
(12:07, 12:02-12:04, E, 5)
(12:08, 12:02-12:04, E, 10)
(12:08, 12:04-12:06, E, 7)
(12:09, 12:04-12:06, L, 17)
```

DZ 3

- vjerojatno su sva ne baš dobra rješenja to je još od prije, mislim da treba paziti na ispitima da u emit isto stavljas onako kako će ti izgledati na kraju izlaz, jedna vrijednost za id i jedna kao value

ZADATAK 1

Korištenjem kolekcije 'dvdrent' i M/R upita potrebno je odrediti i prikazati koliko su puta stanovnici pojedine zemlje iznajmili film te koliko je na njima zaradila iznajmljivačka kuća. U obzir treba uzeti samo filmove koji imaju zapis o uplati, traju duže od 120 minuta te njihov "rental_rate" iznosi više od 3. **Ukupan iznos (zaradu - amount) zaokružiti na tri decimale.**

Nazivi atributa i format se vide iz primjera rezultata:

```
[
  {
    "_id": {
      "country": "Anguilla"
    },
    "value": {
      "count": 1,
      "amount": 4.99
    }
  },
  {
    "_id": {
      "country": "Argentina"
    },
    "value": {
      "count": 2,
      "amount": 9.98
    }
  },
  ... ]
```

```
ZAD1
db.dvdrent.mapReduce(
  function() {
    if(this.payment !== null && this.film.rental_rate > 3 && this.film.length > 120){
      emit( {country: this.customer.address.country}, {amount: this.payment.amount})
    }
  },
  function(key, value){
    var rv = {
      amount: 0.0,
      count: 0
    };
    value.forEach(function(val){
      rv.amount += val.amount;
      rv.count += 1;
    });
    rv.amount = Math.round(rv.amount*1000)/1000;

    return rv;
  }
)
```


ZADATAK 2

Pomoću kolekcije `nobelprizes` doznajte broj laureata i različitih kategorija po godinama, te omjer broja laureata i kategorija po godinama.

Npr. omjer 2018. je iznosio $5/4 = 1.25$ (stvarni podatak se može razlikovati).

Zaokružite omjer na *dvije* decimale i nazovite ga .

`laureateCategoryRatio`

Primjer rezultata:

```
[
  {
    "_id": "2018",
    "value": {
      "laureates": 5,
      "categories": 4,
      "laureateCategoryRatio": 1.25
    }
  },
  ...
]
```

```
ZAD2
db.nobelprizes.mapReduce(
  function() {
    emit(this.category, {ls: this.laureates});
  },
  function(key, values) {
    var rv = {ls: []};
    values.forEach( function(value) {
      rv.ls = rv.ls.concat(value.ls);
    });
    return rv;
  },
  { out: "mr_nobelprizes" },
  {
    finalize: function (key, val) {
      val.ls.sort((x, y) => x.surname.localeCompare(y.surname));
      return val;
    }
  }
);
```

ZADATAK 3

Pomoću kolekcije , doznajte:

- sveukupan zarađeni iznos po zaposleniku
- broj naplaćenih posudbi i zarađeni iznos koje su pojedini zaposlenici naplatili **po pojedinim kategorijama** (`payment` mora postojati).

U slučaju da film pripada više kategorija (npr. "love" i "comedy"), zarada se pripisuje i kategoriji "love" i kategoriji "comedy".

Prikažite ove podatke samo za zaposlenike koji žive u zemlji .

United

Nakon izračunavanja, zaokružite sve decimalne iznose na tri decimale.

Kingdom

Primjer rezultata:

```
[
  {
    "_id": {
      "name": "Diego",
      "lastname": "Diaz"
    },
    "value": {
      "Cartoons": {
        "count": 10,
        "amount": 32.9
      },
      "Drama": {
        "count": 10,
        "amount": 48.901
      },
      "Action": {
        "count": 8,
        "amount": 38.92
      },
      ...
    },
    "totalAmount": 7592.431
  },
  ...
]
```

```

db.dvdrent.mapReduce(
  function() {
    if(this.staff.address.country === "United Kingdom" && this.payment !== null){
      emit({name: this.staff.first_name, lastname: this.staff.last_name},
        {amount: this.payment.amount, categories: this.film.categories});
    }
  },
  function(key, value){

    var rv = {};
    value.forEach(function(val) {
      val.categories.forEach(function(cat) {
        var catname = cat.name;
        if(rv[catname] === undefined) rv[catname] = { count: 1, amount: val.amount };
        else {
          if(val.amount !== 0) rv[catname].amount += val.amount;
          rv[catname].count++;
        }
      });
    });
    for (r in rv) {
      rv[r].amount = rv[r].amount.toFixed(3);
    }

    return rv;
  },
  {
    out: "zad3"
  }
)

```

NESTO

Navedite i objasnite 4 kategorije obrade neograničenih tokova.

→ **A1: vremenski agnostično (time-agnostic)**

- slučaj kada nam vrijeme nije bitno
- osnovne tehnike: filtriranje i spajanje

→ **A2: aproksimativno**

- aproksimacijski algoritmi
- vremenski agnostično, ali generalno imaju ugrađen neki koncept vremena
- osnovne tehnike: sampling, sketching

→ **B1: pomoću prozora u vremenu obrade (processing time windowing)**

- grupiranje zapisa u vremenske prozore s obzirom na vrijeme obrade, događaji su poredani po vremenu dolaska
- nedostatak je da ako nam je bitno vrijeme događaja oni bi trebali pristiti istim redom što se rijetko događa

→ **B2: pomoću prozora u vremenu događaja (event time windowing)**

- grupiranje zapisa u vremenske prozore s obzirom na vrijeme događaja
- veći memorijski zahtjevi radi veličine prozora